

Ygg: 基于树的协作投机解码与仅令牌传输

李思佳^{†‡}、梁雨萌[‡]、李瑞奇[§]、李建江^{†*} 和 吴杰^{‡¶*} [†]北京科技大学, 中国北京 [‡]中国电信云计算研究院, 中国北京 [§]北京大学, 中国北京 [¶]美国天普大学 lisijia@xs.ustb.edu.cn, liangym9@chinatelecom.cn, rqli25@stu.pku.edu.cn, lijianjiang@ustb.edu.cn, jiewu@temple.edu

Abstract——尽管大型语言模型（LLMs）表现出了卓越的性能，LLM推理系统的服务质量（QoS）仍受限于自回归解码固有的较高延迟。为了加速解码过程，最近提出了推测性解码方法，该方法同时使用轻量级草稿模型和位于云端的目标LLM，快速生成候选标记并并行验证这些候选，从而实现每步处理多个标记。然而，将草稿模型和目标模型共同部署在云端会浪费宝贵的GPU内存资源，并限制云端执行并行推理以提升吞吐量的能力。为了通过实现更高吞吐量进一步提高推理QoS，本文提出了Ygg，将草稿模型卸载到成本更低的边缘设备上，以协作方式减轻云端的计算负载。为了解决现有推测性解码中过度数据传输的缺点

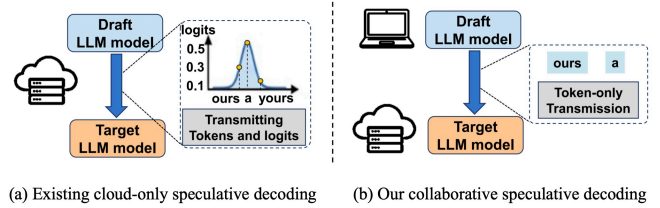


图 1: 先前的推测解码方法与我们的方法示意图。先前的方法在云端部署草稿和目标大语言模型生成候选标记并进行验证。而我们将草稿模型卸载到廉价的边缘设备上，以减轻云端 GPU 负担，并设计了仅传输标记的机制以降低通信延迟。

提出的 [4]–[6]，其中轻量级草稿模型和目标大型语言模型（LLM）都部署在云端。草稿模型快速生成候选标记，而目标模型并行验证这些候选标记，使得多个标记可以在单步中处理。尽管这种草稿-目标模型的协作实现了显著的加速效果，*the additional deployment of the draft model on the cloud consumes valuable cloud GPU memory*, 这限制了大规模并行处理，从而限制了推理吞吐量的进一步提高。

幸运的是，边缘 GPU 的计算能力一直在稳步增长，与云服务器相比，这具有很高的性价比 [7]，从而创造了一个有前景的机会来减轻云端推理的负担 [8]。鉴于草稿模型对计算资源和 GPU 内存的适度需求，本文设想 *whether we can offload the lightweight draft model to inexpensive edge devices, and improve the inference QoS by achieving higher throughput in an edge-cloud collaborative manner*。通过这种方式，可以将云端宝贵的 GPU 内存节省出来，让目标大型语言模型能够并行处理更多样本，从而提高整体吞吐量。

然而，协作式推测解码在实现令人满意的加速时面临以下独特挑战：

(i) *Foremost, in conventional speculative decoding, the large volume of data transmitted between the draft and target*

Index Terms——大型语言模型，协作推理，投机解码，大型语言模型推理的服务质量

一、引言

大型语言模型（LLMs）在推理和文本生成方面表现出了显著的能力 [1], [2]，但LLM推理系统的服务质量（QoS）仍然受到云基础设施上顺序自回归解码所带来的巨大计算和延迟负担的限制 [3]。为了加速解码过程，从而提高LLMs推理的服务质量，*speculative decoding* 最近被

Sijia Li and Yumeng Liang contributed equally to this work.
*: Corresponding authors.

models introduces substantial communication latency. 由草稿模型生成的候选标记及其相关的概率分布 (logits) 需要传输到目标大型语言模型 (LLM)。每个草稿标记的高维 logits 会带来巨大的通信延迟开销, 这甚至超过了协作本身带来的性能提升。尽管一些现有方法[9], [10]试图通过拆分验证阶段或仅传输 top-k 稀疏 logits 来缓解此问题, 高维 logits 仍然是通信延迟的关键瓶颈。如图1所示, 我们的仅标记通信显著减少了传输开销, 从而实现更高的吞吐量提升。

(ii) *Although transmitting fewer tokens can reduce communication latency, the resulting drop in the acceptance rate of candidate tokens degrades the overall acceleration performance.* 投机性解码的加速性能取决于在目标大语言模型 (LLM) i.e. 验证过程中, 草稿模型生成的候选标记的接受概率, 任何被拒绝的标记都必须重新生成, 这会直接影响端到端的加速。当传输较少的候选标记时, 草稿模型的命中率会下降, 导致标记重新生成的频率增加。因此, 在通信开销和接受率之间取得平衡以实现更强的整体加速成为一个关键问题。为了解决这一挑战, 我们将顺序候选标记的生成视为树结构的构建, 并提出了 *budget-aware adaptive tree-based draft generation scheme*。我们不是积极生成大量冗余节点, 而是从全局缓冲区动态选择置信度最高的节点, 并通过预算约束控制节点总数。通过这种方式, 我们可以在传输量较小的情况下实现高候选标记接受率

(iii) *Collaboration between draft generation on the edge and verification on the cloud tends to introduce bubbles, which limits the improvement of overall inference speed.* 在传统的推测解码中, 草稿模型和目标模型以串行方式工作, 从而避免了并行处理导致的过度云端内存占用问题。然而, 在我们的协作模式下, 这种问题不存在, 并且流水线并行模式可以更好地利用双方的资源以实现加速。然而, 边缘端与云端在并行处理过程中存在资源差异和模型差异, 导致执行速度不一致, 产生空闲周期。为了解决这一问题, 我们设计了一种 *asynchronous pipelining* 方法, 根据云端验证延迟调整草稿令牌的数量, 充分利用资源并实现更高的加速。

应对上述挑战的相应方法

集成到名为 *Ygg*¹ 的框架中, 该框架支持基于树的协作式推测解码, 并仅传输令牌。我们在实际边缘云测试床中实现了 *Ygg*, 并通过大量实验评估其性能。实验结果表明, 与云端推测解码方法相比, *Ygg* 实现了 1.94× 的吞吐量提升, 并通过利用廉价的边缘设备将成本降低了 47.3%。与其他最先进的边缘云协作推测解码方法相比, 我们的方法将推理延迟降低了 49.3%, 验证了所提方法的有效性。

我们的贡献总结如下:

- 我们提出了一个通信高效的边缘-云投机解码框架 *Ygg*, 以提高大型语言模型的推理服务质量。它通过引入仅令牌的树状结构投机, 消除了草稿 logits 的传输。
- 提出了一种预算感知的自适应基于树的草稿生成方案, 以在严格的令牌预算下提高候选令牌的接受率。并设计了一种异步流水线机制以提高推理吞吐量。
- 在真实的边缘云测试平台上进行的大量实验验证了 *Ygg* 可以将吞吐量提升最多 1.94× 倍, 并且优于其他现有的协作推测解码方法。

II. 背景与动机

A. Speculative Decoding

大型语言模型 (LLMs) 以自回归方式生成文本, 这通常受到内存限制而非算术密集度的约束 [11], [12]。为加速这一过程, 投机解码 [4]–[6] 引入了一个轻量级草稿模型 M_q 来生成 γ 个候选标记, 然后由目标模型 M_p 并行验证。通过这种方式, 目标模型可以利用轻量级模型快速生成的多个候选标记, 批处理并在一步中生成多个位置的标记。

投机解码的核心机制是如何验证草稿模型生成的候选标记。令 $P_i(x)$ 和 $Q_i(x)$ 分别表示目标模型和草稿模型在第 i 步的概率分布。验证过程包括两个关键阶段:

a) 接受/拒绝: 对于一个草拟的候选标记 x_i , 令 $p_i(x_i)$ 和 $q_i(x_i)$ 表示 $P_i(x)$ 与 $Q_i(x)$ 内的概率值。目标模型评估接受概率:

$$\Pr(\text{accept } x_i) = \min(1, p_i(x_i)/q_i(x_i)). \quad (1)$$

如果被接受, 该令牌将被保留。否则, 它将被拒绝, 流程将转到重新采样阶段。

b) 重新采样: 为了确保输出严格遵循目标分布, 被拒绝的令牌将被来自修正分布的重新采样新令牌 x'_i 替换:

$$x'_i \sim \text{norm}(\max(0, P_i(x) - Q_i(x))). \quad (2)$$

¹以北欧神话中的世界树尤克德拉希命名, 它连接着不同世界的领域。

该机制保证了与目标模型 M_p 相同的分布，只要它能够访问近似的完整条件分布 $Q_i(x)$ 。因此，在传统的投机解码方法中，候选标记以及相应的 logits 会被发送到目标 LLM 模型，如图 1 (a) 所示。该机制在云端产生的延迟很小，但在边缘-云协作推理中却成为了通信瓶颈。

B. The Communication Bottleneck in Edge-Cloud Collaborative Speculative Decoding

在边缘-云协作环境中，将草稿模型放置在边缘端而将目标模型放置在云端是一种有前景的架构 [13], [14]。然而，应用标准的推测解码会产生通信瓶颈。计算式 (2) 需要传输每个草稿 token 的概率分布。对于词汇表大小 V (通常为 32k–128k) 和草稿长度 N ，这会导致 $N \times V$ 的传输负载，通常每步可达到 1MB。在高带宽云集群内部署标准 SD 时 (e.g., 通过 NVLink)，这种数据传输开销可以忽略，但在通过无线网络运行的边缘-云场景中，这成为了一个难以逾越的瓶颈 [15]。尽管最近的优化策略如 DSSD [9] 和 Top-K 稀疏 logits 传输 (TK-SLT) [10] 尝试通过拆分验证阶段或传输稀疏 logits 来缓解这一开销，但它们并未避免传输概率分布。在本文中，

三、方法论

A. Kernel Idea: Naive Sampling and Tree Attention

为了确保生成的输出在分布上与目标模型保持一致而无需访问草稿日志，我们使用 *Naive Sampling* (NS) [16] 进行验证。具体来说，目标模型 M_p 独立计算目标分布 $P_i(x)$ 并相应地采样一个验证令牌 x' 。只有当草稿令牌 x_i 严格匹配 x' 时，才会被接受：

$$\text{Accept}(x_i) = \mathbb{I}(x_i = x'), \quad \text{where } x' \sim P_i(x). \quad (3)$$

尽管高效，NS 的接受率较低，尤其是在每个职位生成的候选人较少时，因为其严格的匹配标准。传输大量候选人会导致高通信开销，因此需要一种策略来平衡接受率和传输成本。

为了在低通信成本下恢复接受率，我们引入了 *tree-based drafting strategy*。草稿模型 M_q 并不是生成单一序列，而是构建一个涵盖多条可能路径的令牌树 $\mathcal{N}$ 。通过使用树注意力 [16]–[18]，目标模型 M_p 可以在一次前向传播中验证整个树。具体来说，拓扑感知掩码 $\mathbf{M}$ 允许查询令牌 u 仅在 v 是 u 的祖先时才关注键令牌 v ：

$$\mathbf{M}_{uv} = \begin{cases} 0, & \text{if } v \in \text{Ancestors}(u) \cup \{u\}; \\ -\infty, & \text{otherwise.} \end{cases} \quad (4)$$

通过应用这种拓扑感知掩码，大小为 $|\mathcal{N}|$ 的树的验证复杂度通过并行计算保持为 $O(1)$ ，确保延迟成本可与验证单条链相媲美。

我们在一个精心设计的边缘-云协作推测解码框架中实现了上述想法。下面，我们将分别介绍框架概览和两个核心设计。

B. Framework Overview

本节介绍了我们为边缘-云协作推测解码设计的通信高效框架 Ygg。为克服标准推测解码中由于高维概率分布传输而导致的关键通信瓶颈，我们提出了一种仅传输令牌的通信协议。通过将设备端的预算感知自适应树构建与流水线式草稿验证机制相结合，该框架实现了快速的边缘-云协作推理。如图 2 所示，该框架主要通过轻量级协议连接两个主要阶段运行：

边缘端草稿。该过程在托管轻量级草稿模型的边缘设备上启动。草稿器不是生成单一序列，而是构建一个标记树以最大化接受率。如草稿面板所示，模型输出标记概率分布 $(p_0, p_1, \dots)$ ，从中抽取多个候选标记。关键是，未立即被选用于当前树扩展的候选标记会保留在全局候选缓冲池中。这些缓冲标记在随后的选择轮中竞争，使得前一步中的高置信度候选可以被重新考虑，从而构建高质量的树拓扑。

仅令牌通信。为了打破带宽瓶颈，我们避免传输概率分布。相反，设备将草拟的树打包成轻量级推测数据包 $\mathcal{P}$ ，仅包含离散整数索引：

$$\mathcal{P} = (\mathbf{t}, \mathbf{p}, \mathbf{pos}), \quad (5)$$

其中 $\mathbf{t}$ 表示标记 ID， $\mathbf{p}$ 表示定义树拓扑的父索引， $\mathbf{pos}$ 表示用于对齐的位置 ID。

云端验证。数据包 $\mathcal{P}$ 被传输到托管大参数目标模型的云服务器。收到后，目标模型基于 $\mathbf{p}$ 构建一个树注意力掩码，以在单次前向传播中计算所有节点的 logits $(q_0, q_1, \dots)$ 。由于草稿 logits 不可用，我们采用 NS 验证策略。云端从根节点开始执行基于路径的动态验证：它从计算的分布中采样目标 token t' ，并将其与草稿子节点进行比较。如果 t' 匹配，验证继续到下一层；否则，过程终止，并返回 t' 。

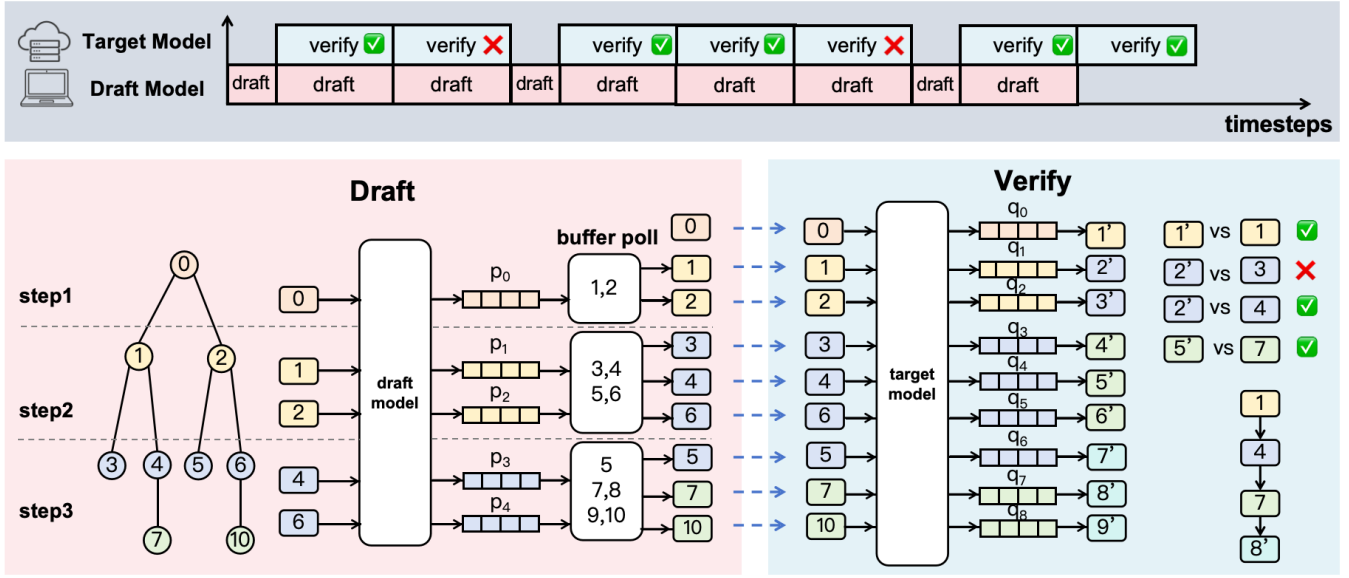


图2: Ygg概览, Ygg通过仅传输令牌的边缘-云协作推测解码。草稿模型在多个步骤中顺序生成不同位置的令牌 (e.g., 黄色节点1-2, 蓝色节点3-6)。在每一步, 会生成多个候选项; 具有较高置信分数的前 k 个会被动态选择, 而其他的保持在全局缓冲池中。只有被选中的令牌最终会被传输到云端, 不带logits, 并与目标LLM的输出 (e.g., 目标2'拒绝草稿3, 而5'接受7) 并行比较。顶部时间轴展示了异步流水线, 其中边缘端在云端验证期间预生成未来令牌, 以掩盖延迟空隙。

作为一个纠正奖励代币。然后云端发送回一个包含已验证路径的响应 $\mathcal{R}$ 。

如顶部时间轴所示, 该框架采用异步执行模型。在云端验证阶段期间, 设备不会闲置; 相反, 它会同时执行下一轮的草稿阶段。如果来自云端的已验证路径与此并行草稿的前缀匹配, 则等待期间的草稿会立即被激活并作为新请求发送, 从而有效掩盖草稿延迟。

C. Budget-Aware Adaptive Tree Construction

为了弥补 NS 接受率下降的问题, 设备必须构建一个高质量的 token 树 $\mathcal{T}$, 覆盖目标模型最可能的输出。我们提出了一种 *Budget-Aware Adaptive Tree Construction* 算法。与现有的方法不同, 现有方法要么缺乏横向扩展 [19], 要么引入过多的计算冗余 [20], 我们的方法利用归一化概率作为置信度分数, 并引入全局候选缓冲区以实现跨层竞争, 从而解决了现有方法中冗余效率低和缺乏横向扩展的问题。

问题表述。先前的研究表明, 草稿模型的输出概率与目标模型的接受概率之间存在显著的正相关。因此 [20], 我们利用草稿模型的预测概率作为置信度的代理, 以估计生成的令牌最终被接受的可能性。设 $\mathcal{V}$ 表示词汇表。给定一个前缀上下文序列

$x_{1:t}$, 草稿模型 M_d 通过 softmax 操作预测下一个标记的概率分布:

$$P(x_{t+1}|x_{1:t}) = \text{Softmax}(\text{Logits}(M_d(x_{1:t}))). \quad (6)$$

我们将 *Token Tree* $\mathcal{T}$ 定义为一组节点, 其中每个节点 u 表示从根节点延伸的一个标记序列 $x_{1:t+d}$ 。考虑到验证过程中固有的顺序依赖性, 即节点 u 的接受需要其所有祖先节点的接受, 路径被目标模型接受的概率近似于沿轨迹所有标记的联合概率。因此, 我们采用累计对数概率 (CLP) 作为度量 $S(u)$ 来量化节点 u 的质量:

$$S(u) = \sum_{k=1}^d \log P(x_{t+k}|x_{1:t+k-1}). \quad (7)$$

我们的目标是构建一棵受严格内存和计算开销限制的树 $\mathcal{T}$, 这些限制由 $|\mathcal{T}| \leq K$ 定义, 其中 K 表示令牌预算。具体而言, 该树旨在包含具有最高 $S(u)$ 值的节点集合, 以最大化草稿模型预测分布中高置信度区域的覆盖, 从而提高推测解码的推测成功率。

算法。现有的树构建算法通常执行按层展开, 在深度 d 选择排名前 k 的节点扩展到 $d+1$, 同时立即舍弃其余节点。我们在边云协同推理的背景下识别出这一范式的两个关键限制。首先, 先生成一个大树然后再修剪以找到最优路径的策略会引入过多的计算开销

Algorithm 1: Budget-Aware Global Adaptive Tree Construction

Input: Prefix $x_{1:t}$, Budget K , Max Depth D , Batch Size B , Branch Width W , Admit Rate N_{limit}

Output: Token Tree $\mathcal{T}$

```

1 Initialize Tree  $\mathcal{T} \leftarrow \{x_{1:t}\}$ ,  $Q \leftarrow \{(0, \text{root})\}$ , Buffer  $\mathcal{B} \leftarrow \emptyset$ ;
2 while depth <  $D$  and  $|\mathcal{T}| < K$  do
3   // 1. Batch Expansion;
4   Pop batch of parents  $\mathcal{P}_{batch}$  from  $Q$  (size  $\leq B$ );
5   Compute logits for  $\mathcal{P}_{batch}$  via Draft Model;
6   for parent  $u \in \mathcal{P}_{batch}$  do
7     Get top- $W$  children  $\{v_1, \dots, v_W\}$  via LogSoftmax;
8     Calculate scores:  $S(v_i) \leftarrow S(u) + \log P(v_i|u)$ ;
9     Add children to Buffer:  $\mathcal{B} \leftarrow \mathcal{B} \cup \{v_1, \dots, v_W\}$ ;
10  // 2. Global Selection (Lateral Comparison);
11  Sort  $\mathcal{B}$  descending by score  $S(\cdot)$ ;
12   $N_{admit} \leftarrow \min(|\mathcal{B}|, N_{limit}, K - |\mathcal{T}|)$ ;
13   $\mathcal{C}_{best} \leftarrow$  Top  $N_{admit}$  nodes from  $\mathcal{B}$ ;
14  Remove  $\mathcal{C}_{best}$  from  $\mathcal{B}$ ;
15  // 3. Tree Update;
16  for node  $v \in \mathcal{C}_{best}$  do
17    Add  $v$  to  $\mathcal{T}$ ;
18    Push  $v$  to  $Q$  for future expansion;
19  if  $N_{admit} = 0$  then
20    break;
21 return  $\mathcal{T}$ 
  
```

冗余。在资源受限的边缘设备上，为最终将被剪枝的节点计算 logits 会造成大量计算资源的浪费。其次，置信度衰减表现为两个维度：深度（随着序列长度增加，概率下降）和宽度（在同一层中，Rank-2 令牌的概率低于 Rank-1 令牌）。现有方法倾向于优先考虑深度，通常仅扩展最优分支。然而，位于浅层的高概率节点可能具有比位于深层、置信度已经降低的节点更高的全局可能性 $S(u)$ 。忽略这些浅层节点会导致树结构无法实现全局最优性，如固定形状和按层扩展中所示（图 3(a) 和 (b)）。

为了解决这些缺点，我们建议通过全局缓冲池实现横向扩展，从而实现图 3(c) 所示的全局自适应扩展。我们不再将比较限制在同一层级内，而是允许浅层的候选节点与深层的节点直接竞争。因此，如果一个深层分支进入低置信度区域，算法会动态地转向展开以前缓冲的浅层高概率节点。

详细的过程在算法1中有说明，它

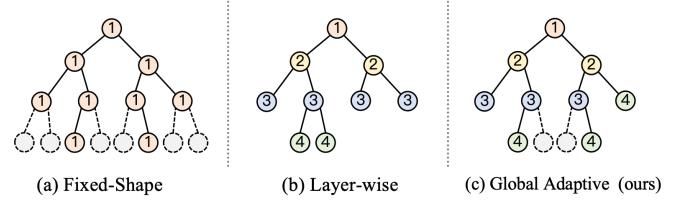


图3：树扩展策略比较。(a) 固定形状扩展首先积极扩展成一个大且规则的树，然后再进行剪枝，浪费计算资源。(b) 分层扩展在每一步只选择当前层中最优的节点。(c) 我们的全局自适应扩展从全局缓冲区动态选择最高置信度的节点，优化生成的令牌树结构。

通过两个主要组件协调生成过程：一个 *Global Candidate Buffer* ($\mathcal{B}$) 和一个 *Priority Queue* (Q)。该过程从初始化包含输入前缀的树 $\mathcal{T}$ 开始，同时用根节点为 Q 提供初始值以调节扩展顺序。在每个迭代循环中，从 Q 获取一批父节点，以通过草稿模型进行并行前向计算。对于每个父节点，计算前 W 个子令牌的概率。关键在于，这些潜在节点不会立即附加到树中，而是与它们的累积分数 $S(u)$ 一起存入全局候选缓冲区 $\mathcal{B}$ 。随后，采用全局选择机制，将整个缓冲区——包括新生成的候选节点和前一层未选中的选项——按分数排序。前 N_{limit} 个节点被加入 $\mathcal{T}$ 并从 $\mathcal{B}$ 移动到 Q 以供后续扩展，而未被选中的节点则保留在 $\mathcal{B}$ 中。这实现了横向扩展

D. Asynchronous Pipelining

虽然纯索引协议可以最小化传输延迟，但目标模型验证中固有的计算延迟是不可避免的，从而产生一个空闲窗口（或“气泡”），边缘设备在此期间等待响应（图 4）。为了弥补这一差距并充分利用边缘计算资源，我们提出了一种 *Batched Pre-Drafting* 机制。该方法使边缘设备能够在等待验证的同时，推测性地生成下一轮的潜在草稿。在收到云端响应后，如果最终确定的路径与等待期间推测性扩展的分支一致，系统可以跳过下一轮的起草阶段，直接传输预计算的树。整体过程总结见算法 2。

候选树初始化。我们不只是简单地加深当前的标记树 $\mathcal{T}_{curr}$ ，而是选择在 $\mathcal{T}_{curr}$ 的高置信叶节点上预先构建独立的 *Candidate Trees*。这一策略确保同时生成的候选树在结构上与原本将生成的草稿 *post-verification* 对齐，有效地

Algorithm 2: Batched Pre-Drafting with Asynchronous Verification

Input: Current tree $\mathcal{T}_{curr}$, draft model $\mathcal{M}_d$, cloud model $\mathcal{M}_c$, top- k k

- 1 send $\mathcal{T}_{curr}$ to Cloud;
 - 2 **async (Cloud):** $(\mathcal{R}, l_{acc}) \leftarrow \text{Verify}(\mathcal{T}_{curr}, \mathcal{M}_c)$;
 - 3 **async (Edge):** $\mathcal{T}' \leftarrow \text{Expand}(\mathcal{T}_{curr}, \mathcal{M}_d, 1)$;
 - 4 $\mathcal{L} \leftarrow \text{TopKLeaves}(\mathcal{T}', k)$;
 - 5 $\mathbb{S} \leftarrow \{\text{InitTree}(l) : l \in \mathcal{L}\}$;
 - 6 **repeat:** $\mathcal{B} \leftarrow \text{Frontier}(\mathbb{S})$;
 - 7 $\mathbb{S} \leftarrow \text{BatchExpand}(\mathcal{B}, \mathcal{M}_d)$;
 - 8 **until** $\mathcal{R}$ received;
 - 9 $l_{acc} \leftarrow \text{AcceptedLeaf}(\mathcal{R})$;
 - 10 $\mathcal{T}_{next} \leftarrow \begin{cases} \text{Retrieve}(\mathbb{S}, l_{acc}), & l_{acc} \in \mathcal{L} \\ \text{StandardDraft}(l_{acc}, \mathcal{M}_d), & \text{otherwise} \end{cases}$;
 - 11 send $\mathcal{T}_{next}$ to Cloud if $l_{acc} \in \mathcal{L}$;
-

预测云端的接受决策。此并行化旨在将下一轮的草拟与当前轮的验证同步，从而消除云端等待边缘设备草拟阶段所产生的空白。令 $\mathcal{L} = \{l_1, l_2, \dots, l_N\}$ 表示 $\mathcal{T}_{curr}$ 中的叶节点集合。在将 $\mathcal{T}_{curr}$ 传输到云端后，边缘设备在 $\mathcal{T}_{curr}$ 上执行单步扩展——考虑到云端可能返回的奖励 token——以获得 $\mathcal{T}'_{curr}$ 。随后，从中选择一部分高置信度叶节点 $\mathcal{L}_{top} \subset \mathcal{T}'_{curr}$ 作为下一步推理的潜在起点。对于每个选定的叶节点 $l_i \in \mathcal{L}_{top}$ ，初始化对应的候选树 $\mathcal{T}'_i$ ，其中 $\mathcal{T}'_i$ 的根逻辑上连接到 l_i ，表示在 l_i 被接受的情况下假设的路径延续。

批处理扩展与激活。为了在资源受限的边缘设备上高效地同时构建多个候选树[21]，我们利用了草稿模型的 *Batch Processing* 能力。在每个深度 d 的扩展过程中，来自所有候选树的候选节点 $\{\mathcal{T}'_1, \dots, \mathcal{T}'_k\}$ 会被聚合到单个推理批次中。这种批处理方法分摊了与内核启动和内存 I/O 相关的开销[22]，确保同时构建 k 候选树的总延迟显著低于顺序执行。当收到来自云端的验证响应 $\mathcal{R}$ 时，边缘设备会立即执行匹配操作。如果被接受的路径在叶节点 $l^* \in \mathcal{L}_{top}$ 处终止，则相应的候选树 $\mathcal{T}'_{l^*}$ 会立即“激活”。这个预计算的树会被立即序列化并传输到云端，有效地消除了下一轮（即 $T_{draft} \rightarrow 0$ ）的草稿延迟。如果接受的路径

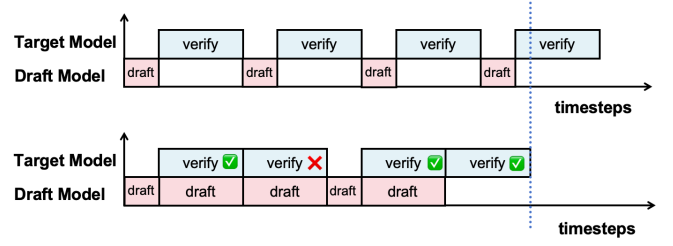


图4: 标准顺序推理（上）与我们的异步流水线（下）的时间线比较。在标准方法中，草稿模型在目标模型进行验证时处于空闲状态。我们的方法利用这一间隔进行预草稿，有效地掩盖了验证延迟。

四、评估

实验设置。我们在异构边缘-云协同计算环境中评估所提出的框架。云服务器配备八块 NVIDIA A100 GPU (40G B)，承载大规模目标模型 (M_p)，代表高性能的集中式基础设施，而边缘节点为一台消费级工作站，配备两块 NVIDIA RTX 4090 GPU (24GB)，运行轻量级草案模型 (M_q)，模拟典型的边缘设备。系统使用 PyTorch [23] 实现，边缘节点与云节点之间的所有交互均通过 *gRPC* 进行，以最小化通信开销并确保可靠传输。所有实验均在广域网 (WAN) 设置下进行，最大带宽为 200 Mbps，以模拟真实部署环境。为了评估所提方法的泛化能力和有效性，我们在多个模型对和标准基准上进行了广泛评估。

评估指标。关键评估指标是系统的 *token throughput*，正式定义如下：

$$\text{Throughput} = \sum_{i=1}^{\text{Batch Size}} \text{Token-Length}_i / \text{Inference-Latency} \quad (8)$$

它通过将生成的令牌总数（对应于每个并行请求生成令牌长度的批量大小）除以完成这一批次推理任务所消耗的总延迟来计算。具体来说，批量大小会自适应设置，以占用云端固定的 GPU 内存，从而在边云协同和仅云场景下实现不同设置之间的公平比较。

除了吞吐量，我们还根据现行市场租赁价格量化服务 *costs*：A100 40G GPU 为每小时 4.05 美元，RTX 4090 GPU 为每小时 0.35 美元 [24], [25]。在消融研究中，我们使用 *accept rate* 验证了基于树的草稿算法的有效性，*accept rate* 是最终生成的 token 中由草稿模型生成并被目标大语言模型接受的比例。

基线。为了全面评估我们所提出框架的有效性，我们将其与 *two cloud-only* 推理方法以及 *four state-of-the-art collaborative* 进行比较

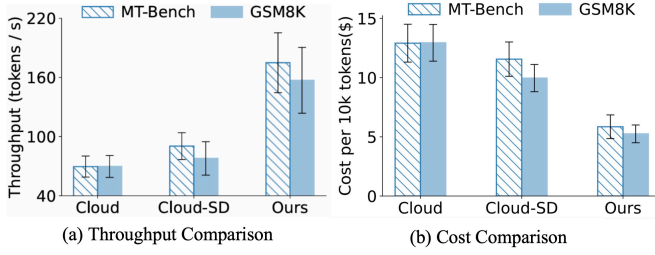


图 5: 在 MT-Bench 和 GSM8K 数据集上, 云端仅 LLM 推理与我们方法的比较。

表 I: 在云存储和延迟方面的比较。

Method	Cloud Model Memory	Latency	Throughput
Cloud	121 GB	16.4 s	69.5 tokens/s
Cloud-SD	146 GB	5.36 s	90.3 tokens/s
Ours	121 GB	6.20 s	175 tokens/s

方法。两个典型的纯云方法如下: (1)

Cloud-Only Autoregressive Generation, 其中目标模型完全在服务器上执行, 作为推理延迟的标准基准。(2)

Cloud-Only Speculative Decoding (Cloud-SD) [5], 其中草稿模型和目标大语言模型都部署在云端以执行预测解码。我们将 Cloud-SD 的批量大小配置为 5 以确保稳定执行。

然后我们选择了四种最先进的协作方法。(1)

Model Split [26], 一种传统的卸载策略, 将模型层在边缘设备和云之间进行划分。(2)

Distributed Speculative Decoding (DSD) [13], 在分布式环境中应用标准投机解码, 需要传输完整的概率分布。(3)

DSSD [9], 通过拆分验证阶段优化协作投机解码的通信延迟。(4) *TKSLT* [10], 通过仅传输前k个稀疏logits来缓解协作投机解码中的通信瓶颈。

模型和数据集。为了展示我们框架在不同参数规模下的可扩展性, 我们使用 Vicuna 系列模型 [27] 进行评估, 这些模型是在 LLaMA 模型 [2] 的基础上微调的。我们建立了两个不同的草稿-目标配对: Vicuna-33B 配对 Vicuna-7B, 以及 Vicuna-7B 配对 Vicuna-68M。在硬件分配方面, 33 B 模型分布在 8 块 GPU 上以适应其巨大的内存占用, 而 7 B 模型和 68M 模型则部署在单块 GPU 上。我们在两个广泛使用的基准测试上评估这些模型, 以覆盖不同的能力, 具体使用 MT-Bench [28] 进行多轮对话和指令遵循评测, 以及 GSM8K [29] 进行数学推理任务评测。对于 GSM8K, 我们仅使用前 100 个测试样本进行评估。除非另有说明, 所有实验配置均采用统一的温度设置为 1.0, 并将最大生成长度设置为 128 个标记。

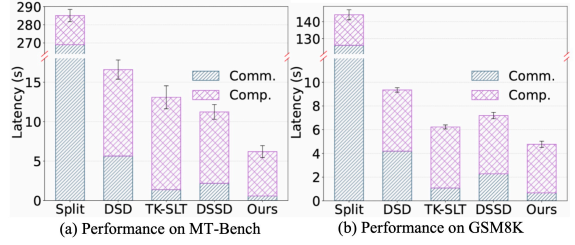


图6: 在使用33B-7B模型的两个数据集上, 不同协作方法的通信和计算延迟。

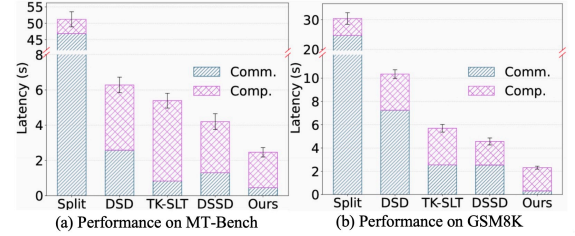


图7: 7B-68M模型的延迟性能。

A. End-to-End Performance and Cost-Efficiency

我们首先评估所提出框架的端到端性能, 并与仅云端的基线进行比较, 以展示释放云端内存以最大化吞吐量的核心优势。结果如图 5 和表 I 所示。

如表 I 所示, 在云端同时部署草稿 7B 模型和目标 33B 模型 (Cloud-SD) 需要 146 GB 的云内存。通过将草稿模型卸载到边缘设备, 我们的方法将云内存占用减少回 121 GB, 与标准自回归生成 (Cloud) 的内存使用量相当。关键在于, 这部分节省的内存允许云服务器容纳更大的批量大小, 这转化为巨大的吞吐量优势: *our approach achieves a higher throughput of 175 tokens/s, which is $2.52\times$ of cloud-only autoregression and $1.94\times$ of Cloud-SD*。这验证了, 尽管固有的云-边缘通信延迟使我们的方法相比 Cloud-SD 略慢 (6.20 秒 vs. 5.36 秒), 但显著的内存节省带来了卓越的吞吐量收益。

图5进一步说明了这一优势在不同数据集上的表现。与 Cloud-SD相比, 我们的边缘-云协作方法 *consistently achieves about $2\times$ throughput gains* 在 MT-Bench 和 GSM8K 数据集上均表现良好。同时, 由于草稿模型的计算被卸载到成本效益高的边缘设备上, *our method separately reduces the token generation cost by about 47.3% and 31.0% on each dataset*。这表明我们的方法作为一种经济的快速大规模 LLM 服务解决方案具有很大潜力。

B. Latency Breakdown and Collaborative Efficiency

为了与现有的边缘-云协作方法进行全面比较, 我们将总推理时间分解为通信延迟和计算延迟。比较结果

表 II: 消融研究。

Method	Throughput (token/s)	Cost _{100k} (\$)
w/o Tree	143.70	6.51
w/o Parallel	148.56	6.30
Full	174.94	5.35

针对 Split、DSD、TK-SLT 和 DSSD 的情况如图 6 和图 7 所示，使用了 33B-7B 模型和 7B-68M 模型。

通信延迟。由于大语言模型（LLM）中间特征较大，分割（Split）方法产生的传输量比其他方法高出数十倍，这严重影响了相应的延迟。其他协同推测解码方法都受到 logits 传输带来的大量通信开销的影响。相反，

our token-only communication

protocol consistently reduces the communication latency by over 38.4% across diverse datasets using LLM models of varying sizes. 因此，我们的方法在整体延迟上明显优于所有最先进的方法。

计算延迟。除了通信节约之外，我们的方法还展示了优越的计算延迟。在 MT-Bench 数据集上，由于任务更具挑战性且计算复杂度更高，这种优势变得更加明显。具体来说，*compared to the state-of-the-art methods, our computation latency of 33B-7B model and 7B-68M model are reduced by 37.9% and 30.5%, respectively.* 这归因于两个关键设计。首先，我们的预算感知树草稿策略显著提高了候选令牌的接受率，从而减少了边缘与云之间所需的总交互轮次。其次，我们的异步流水线机制有效地将边缘草稿模型的计算时间与云端的验证时间重叠，从而通过最小化空闲间隙掩盖了这种计算开销。

因此，我们的方法在不同任务复杂度（MT-Bench 和 GS M8K）和不同模型规模（33B-7B 和 7B-68M）下始终实现了最低的整体延迟，并且 *reduces the overall inference latency by up to 49.3%.* 这验证了其在真实边缘云环境中的强大泛化能力。

C. Ablation Study

为了全面理解每个组件的贡献以及在各种超参数下的性能，我们进行了详细的消融研究和评估。

核心组件的有效性。我们首先消融了框架的两个基本组件：基于树的草稿结构和异步流水线机制。如表 II 所示，完整的 Ygg 框架实现了最高的吞吐量 174.94 令牌/秒和最低的推理成本。没有树结构的配置（w/o Tree）表现出严重的性能下降，仅实现 143.70 令牌/秒的吞吐量。这种显著的退化表明，没有基于树的结构，系统会退化为脆弱的单序列草稿范式。这种线性

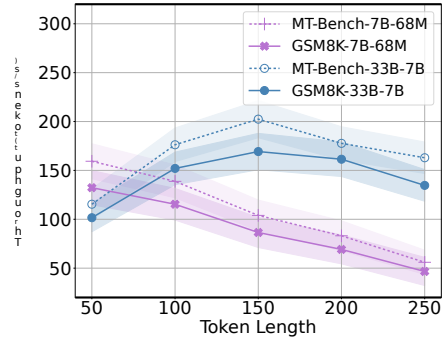


图 8: 不同序列长度下的性能。

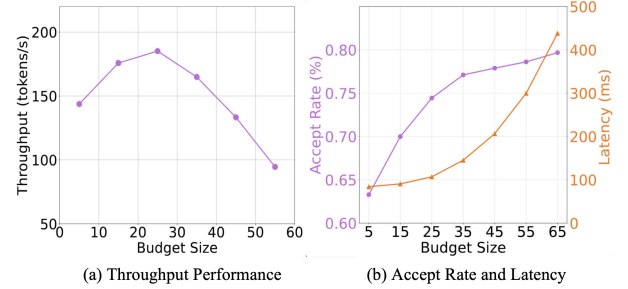


图 9: 预算规模选择的影响。随着预算的增加，接受率上升，但传输延迟也增加，因此吞吐量先增加后减少。

该方法未能利用云的并行验证能力，导致令牌接受率较低且通信轮次增加。此外，在没有并行执行的配置下（w/o Parallel）速度降至 148.56 个令牌/秒。这个性能差距显示在标准顺序执行下，云验证阶段存在计算空闲的“气泡”。通过利用我们的算法预先起草候选树，完整框架可以有效地将起草与验证重叠，从而掩盖延迟并最大化吞吐量。

序列长度的影响。如公式8所示，吞吐量受到生成标记长度的影响。我们分别将最大标记长度设置为不同的值，并在图8中绘制了吞吐量。观察到对于大型LLM模型，如 i.e.、33B-7B，在不同序列长度下吞吐性能保持在150 tokens/s以上的稳定水平。而对于7B-68M模型对，则随着序列长度的增加，吞吐量呈持续下降趋势。其根本原因在于68M草稿模型的释放节省的GPU内存相对较少，这几乎无法为长序列获得足够的批量大小提升。This implies that our approach yields more stable acceleration for larger LLM models, which releases substantial GPU memory. 随着模型复杂度的不断增加，预计其在未来将发挥更重要的作用。

预算规模的影响。图 9 展示了树预算规模引入的权衡。随着预算的增加，边缘设备可以传输更多的候选令牌。这提高了接受率，如图 9 (b) 所示。然而，

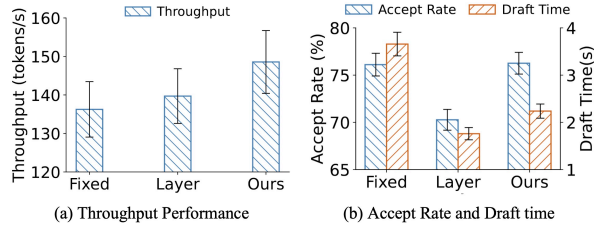


图 10: 树构建算法的影响。

扩展的预算同时增加了单次交互轮次（起草、验证和通信）的计算和传输延迟。因此，如图9(a)所示，*the throughput exhibits a concave trend: it initially increases as the benefits of a higher acceptance rate dominate, but eventually decreases when the per-round latency overhead outweighs the reduction in interaction rounds.*

树构建算法的影响。图 10 将我们的预算感知全局自适应树构建算法与标准的固定和按层扩展算法进行了比较。*Fixed* 策略在剪枝前大胆地扩展一个大型的规则树；虽然这种穷尽式的方法可以获得最高的令牌接受率，但其吞吐量受到边缘设备上巨量冗余草稿时间开销的瓶颈限制。相反，*Layer-wise* 方法的接受率明显较低，因为它严格限制节点选择在当前层内，频繁丢弃位于较浅深度的全局最优节点。我们提出的全局自适应算法有效地解决了这一困境。通过从全局缓冲中动态选择高置信度节点，它在保持可接受计算冗余水平的同时，维持了高度竞争的接受率。因此，它实现了更好的权衡，并在三种策略中提供了最高的整体吞吐量。

五、相关工作

高效的设备端推理。为了减轻大模型在集中式云部署中的巨大开销，近期研究转向了支持本地推理。然而，现有的权宜之计无法在不做重大妥协的情况下解决这一问题。具体而言，权重量化方法 [30], [31] 成功压缩了内存占用，但通常会带来生成质量的不可逆下降。另一种方法是用边缘原生轻量模型替代它们以实现本地执行 [32], [33]，但其推理能力远不及大规模模型。其他方法，如存储卸载 [34]，尝试通过系统 RAM 或 SSD 绕过 GPU 内存限制。但它们受到严重的 I/O 带宽瓶颈影响，往往导致延迟高于纯云解决方案。此外，分布式边缘推理 [11], [35] 将模型分布在多个设备上，但在很大程度上受到障碍的影响。

投机性解码。投机性解码已经成为一种有前景的范式，用于加速自回归生成，同时保持与目标大语言模型的无损一致性 [4]–[6]。基本的投机性解码过程使用轻量级的草稿模型快速生成候选序列，以便目标模型进行并行验证。为了提高草稿效率和接受率，最近的进展多样化了草稿机制，探索了自投机解码 [36]、基于检索的方法 [37] 以及无草稿生成架构 [18]。此外，最先进的方法广泛采用基于树的解码 [16]、[20]、[38]、[39]，以同时探索多条生成路径，最大化每次验证的产出。然而，这些先进的投机性解码技术本质上是为仅云端基础设施设计的，这类基础设施配备了共享内存和高带宽互连，使其无法应用于边缘-云协同环境。

边缘云协同推断。为了缓解云服务器在LLM推断中的堆积，边缘云协作已被积极研究。早期协作范式主要依赖分流计算[40]。虽然对卷积神经网络（CNN）有效，但由于持续传输大量中间作用张量，LLMs会带来不可接受的通信开销。为了完全规避张量传输，协同推测解码通过将草稿模型卸载到边缘，同时保留目标模型在云端，扩展了上述推测解码范式[13], [14]。然而，它引入了一个新的通信瓶颈，要求在受限网络中持续传输已草拟的令牌及其高维概率分布（logits）[9]。即使采用了像Top-K稀疏Logit这样的优化协议[10]，传输这些重负载依然难以实现。

六、结论

在本文中，我们提出了Ygg，一种高效的边缘-云协作推测解码框架，具有较小的传输开销，以提升大型语言模型（LLM）推理的服务质量。通过将轻量级草稿模型卸载到低成本的边缘设备，并采用仅传输令牌的机制，Ygg成功解决了纯云端设置中严重的GPU内存限制问题以及现有协作方法中存在的重通信瓶颈。在该框架中，我们引入了两个关键机制：一种预算感知的全局自适应树状草稿生成方案，能够在最小开销下保证高令牌接受率；以及一种异步流水线策略，实现边缘草稿与云端验证的重叠，从而消除计算空闲气泡。大量实验验证了其能够有效提高吞吐量并降低LLM推理成本。

REFERENCES

- [1] DeepSeek-AI, “DeepSeek-V3 技术报告”, DeepSeek-AI, 技术报告, 2024, arXiv 预印本 arXiv:2412.19437. [2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: 开放高效的基础语言模型”, *arXiv preprint arXiv:2302.13971*, 2023. [3] R. Pope, S. Douglas, A. Chowdhery *et al.*, “高效扩展 Transformer 推理”, 发表于 *Proceedings of Machine Learning and Systems (MLSys)*, 第 5 卷, 2023. [4] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, 和 Z. Sui, “投机解码: 利用投机执行加速 seq2seq 生成”, 发表于 *Findings of the Association for Computational Linguistics: EMNLP*, 2023, 第 3909–3925 页. [5] C. Chen, S. Borgeaud, G. Irving *et al.*, “通过投机采样加速大型语言模型解码”, *arXiv preprint arXiv:2302.01318*, 2023. [6] Y. Leviathan, M. Kalman, 和 Y. Matias, “通过投机解码快速从 Transformer 推理”, 发表于 *Proceedings of the International Conference on Machine Learning (ICML 2023)*. PMLR, 2023, 第 1927 4–19286 页. [7] P. SK, S. A. Kesanapalli, 和 Y. Simmhan, “表征性能的
- [20] 王杰, 苏勇, 李军 *et al.*, “Opt-tree: 基于自适应草稿树结构的投机解码”, *Transactions of the Association for Computational Linguistics*, 第13卷, 第188–199页, 2025年. [21] 郑毅, 陈勇, 钱斌, 史翔, 舒毅, 陈杰, “边缘大语言模型综述: 设计、执行与应用”, *ACM Computing Surveys*, 第57卷, 第8期, 第1–35页, 2025年. [22] 余国一, 郑在成, 金光伟, 金胜, 秦勃光, “Orca: 基于Transformer生成模型的分布式服务系统,” 载于 *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022年, 第521–538页. [23] Paszke A., Gross S., Massa F., Lerer A., Bradbury J., Chanan G., Killeen T., Lin Z., Gimelshein N., Antiga L. *et al.*, “PyTorch: 一种命令式、高性能深度学习库,” *Advances in neural information processing systems*, 第32卷, 2019年. [24] Google Cloud, “Google云定价,” <https://cloud.google.com/>, 2026年, 访问: 2026年. [25] Vast.ai, “Vast.ai全球GPU市场定价,” <https://vast.ai/>, 2026年, 访问: 2026年. [26] 江磊, 傅思德, 朱勇 *et al.*, “Janus: 协作视觉变换器